

Exercise Key — Ship From Slack

Answers, marking notes and the pre-flight checklist

Do not distribute. Contains the answers to Parts 3, 5, 7 and 9.

Read the pre-flight checklist before running this with a cohort — the reference repo has credentials committed in `harness/tick.sh` that must be rotated first.

Module 14 Exercise — Facilitator Key

Do not hand this file to learners. It contains the answers to Parts 3, 5, 7 and 9, plus the marking notes. The learner document is `Module-14-Exercise-Ship-From-Slack.pdf`.

Before you run this with anyone

- ☐ **Rotate the credentials committed in `harness/tick.sh`.** That file, added in commit `6a0e9e5` on 22 June 2026, contains a Notion API key, a Slack user token, and a Gmail client secret plus refresh token, pasted directly into the script. `.gitignore` correctly excludes `.env` — the keys bypassed that protection by living in a tracked file. Rotate all four, then decide what the class sees.
- ☐ **Decide which fork the class gets.** Two options, and they teach different things:
 - *Sanitised fork* — you strip `tick.sh`, learners never see live keys, Part 9 still works because the git history and the hardcoded bot ID remain. Safest, and the default choice for an open cohort.
 - *As-is, with access controlled* — learners find real leaked keys in a real repo, which is the most memorable version of video 6's lesson. Only do this **after** rotation, and say out loud on day one that the keys are dead.
- ☐ **Pin a commit.** Give the class one SHA to check out, so everyone hits the same traps and your answer key stays true.
- ☐ **Grant access.** The repo is private. Nobody can start until you have added them or published a copy.
- ☐ **Check your own Slack workspace policy.** If app installs need admin approval, get the class's apps approved in advance or the first two hours evaporate.

Answer key

Part 3 — the title-property trap

The repo disagrees with itself about what the Notion title column is called:

File	Expects the title property to be
<code>harness/skills/poll-slack.md</code>	<code>Task name</code>
<code>harness/tick.sh</code>	<code>Deliverable</code>

Neither is provisioned by the setup skill — it only adds `Agent Session ID`, `Last Agent Update`, `GitHub PR` and `Slack Thread`, and correctly notes that `Status` is built in. So the learner's own database has whatever title Notion gave it (usually `Name`), and **one of the three has to change**.

Full marks: they found the disagreement, named the file(s) they edited, and chose one name consistently.

Half marks: it works, but they only patched the file that happened to break first.

Part 5 — why nothing happens until they edit the poller

`harness/skills/poll-slack.md` is written for one person's workspace. Two things block every learner:

1. Its search query is a **hardcoded bot user ID** from the original workspace. Until they replace it with their own bot's `U...` ID, `search.messages` matches nothing. 2. Its "what counts as actionable" rule filters for messages addressed to one named person and their job title. Learners must decide their own rule. Any defensible rule earns the marks — the point is that they noticed a business rule buried in a poller.

Also expect these three stalls, in order of frequency:

- **`last_sync_at` is newer than their test message.** They post, then set up, then wait. Fix: post a fresh message.
- **Bot token used for search.** `search.messages` requires a **user** token with `search:read`; the bot token returns `not_allowed_token_type`. This is the one detail the module's videos do not cover, which is why Part 1 calls it out separately.
- **Held tick lock** after they Ctrl-C a tick mid-run. The next tick refuses for 30 minutes and they conclude the harness is broken.

Part 7 — expected error strings

#	Expected	Notes for marking
1	HTTP 404 <code>object_not_found</code>	Accept the message body too: "has not been shared with your integration"
2	<code>not_in_channel</code>	Some SDKs surface it as <code>channel_not_found</code> for private channels the bot cannot see — accept either with an explanation
3	<code>missing_scope</code>	The response names the scope required; full marks if they quote it
4	Tick refuses, lock held	Accept either fix: clearing the row, or waiting out the 30-minute staleness window

Part 9 — the contradictions that are actually there

They need three; there are at least five. One **must** be about credentials.

1. **Credentials in a tracked script** (`harness/tick.sh`). Contradicts video 6 directly. The sharp version of this answer notices that `.gitignore` *does* cover `.env` — the protection was right and the practice went around it. 2. **A person hardcoded into shared tooling.** The Slack poller filters for one named individual; the README's rules require commits prefixed with one person's name and work kept inside their folder. Contradicts nothing in the videos, but it is the reason the thing did not work for them, and naming it is worth the marks. 3. **Notion-Version: 2022-06-28 throughout.** Video 4's answer: that version pre-dates the 2025-09-03 split of databases into data sources, so the code queries `/v1/databases/:id/query`. Nothing breaks *today* — old pinned versions keep working, which is exactly why pinning matters. It breaks the moment someone bumps the header without moving to `data_source_id`, or the moment a database gains a second data source and a `database_id` stops being precise enough to address it. 4. **The title-property disagreement** from Part 3. 5. **Docs and code drift.** `SETUP.md` describes polling Notion, Slack and GitHub; `harness/CLAUDE.md` adds Gmail; `tick.sh` is a standalone script that polls Gmail and Slack and does not touch the SQLite state the rest of the system depends on. `SETUP.md` also references a dev server on port 8001 belonging to a different project.

Accept any well-argued finding not on this list. Reject "the code is messy" without a specific file and a specific video.

Marking notes

The section that actually separates people is "What broke and how I found it." A learner who ran nothing but `/setup` and got lucky writes three vague lines. A learner who understands the system writes about the chain: which link they checked first, what the database said, and how they knew it was a permission problem rather than a code problem. Mark generously for a debugging narrative with evidence, including one from someone who never got the full chain working.

A clean run with no story is suspicious. Ask them to reproduce error 4 live.

Zero-tolerance rule: a live credential visible anywhere in the submission voids the exercise (–100) and the learner rotates it that day. State this on day one, not at marking time.

If a learner cannot get the chain working at all, full marks are still reachable at 60 via E1–E4, E7, E8 and E9. Say that up front — it stops people faking E5.

Timing (3 hours)

Part	Planned	Where cohorts actually lose time
1 · Three doors	25 min	Slack admin approval, if not arranged in advance — can cost a whole day
2 · Fork + <code>/setup</code>	30 min	Usually fine; the wizard's validation catches problems early
3 · Notion board	20 min	The title trap — budget 30 min for a first cohort
4 · First tick	20 min	Learners read "0 events" as failure. Say in the briefing that it is a pass
5 · Order from Slack	40 min	The big one. The bot ID and <code>last_sync_at</code> . Expect to rescue at least a third of the room
6 · Into GitHub	30 min	Fine once Part 5 works
7 · Break it	25 min	Enjoyable; people finish early
8 · Make it safe	20 min	—
9 · Read critically	20 min	Weak submissions stop at one finding; prompt them toward the git history

The ten-minute rescue for Part 5, in order: is the loop running · is `last_sync_at` after your message · did you change the bot ID · are you searching with the user token · is the lock held · does the title property match.

Two questions worth putting to the room afterwards

1. This harness polls every 180 seconds. What did you gain by polling, and what would you have to build and run if you switched to webhooks? (Ties back to video 5's doorbell.) 2. The context key is always the Notion ticket, never the Slack thread. Why? What breaks if you key on the thread instead? (The answer — one conversation can span Slack, a PR and a ticket, and only one of those survives as the durable record — is the whole of video 1 in one design decision.)